

Data Engineer Interview Prep

Week 1 & 2 — PySpark, Spark SQL & GCP

Table of Contents

Data Engineer Interview Prep — Week 1 & 2 (PySpark, Spark SQL, GCP)..**Error! Bookmark not defined.**

PART A — WEEK 1: Spark Ecosystem, Architecture & RDDs	1
Section 1 — Spark Fundamentals & Ecosystem	1
Section 2 — RDDs (Resilient Distributed Datasets)	3
Section 3 — Shared Variables.....	5
Section 4 — Cluster Deployment (Cluster Manager, EMR, Dataproc, spark-submit).....	6
Section 5 — GCP Foundations (Console/CLI, Cloud Storage, IAM).....	8
PART B — WEEK 2: Spark SQL, DataFrames, Datasets & BigQuery.....	9
Section 6 — Spark SQL & Entry Points	9
Section 7 — DataFrame Operations.....	10
Section 8 — Joins	11
Section 9 — Partitioning, Sorting, Bucketing & Caching.....	12
Section 10 — BigQuery & Spark-vs-BigQuery Decisions.....	13

PART A — WEEK 1: Spark Ecosystem, Architecture & RDDs

Section 1 — Spark Fundamentals & Ecosystem

Q1. ● What is Apache Spark, and why did it become popular over Hadoop MapReduce?

Spark is a distributed, in-memory data processing engine for large-scale data. MapReduce writes intermediate results to disk between every map and reduce stage; Spark keeps intermediate data in memory and only spills to disk when required. For iterative workloads (machine learning, graph algorithms) and interactive queries, this can be 10–100x faster.

Real-world example: A retail client re-ran a customer-segmentation model 20 times a day. On MapReduce each run re-read the dataset from disk, taking ~45 minutes. Rewritten in Spark with the dataset cached in memory, subsequent iterations dropped to ~4 minutes because the data never left RAM.

Q2. ● Describe the main components of the Spark ecosystem.

- **Spark Core** — the engine: task scheduling, memory management, fault recovery, and the RDD API.
- **Spark SQL** — structured data via DataFrames/Datasets and SQL.
- **Spark Streaming / Structured Streaming** — near-real-time processing of data streams.
- **MLlib** — distributed machine learning library.
- **GraphX** — graph and graph-parallel computation.

Everything sits on top of Spark Core, which is why the same cluster can run a batch ETL job and a streaming job.

Q3. ● Compare Hadoop and Spark across storage, processing, and speed.

Dimension	Hadoop (MapReduce)	Spark
Storage	HDFS (its own storage layer)	No storage layer; reads from HDFS, S3, GCS, etc.
Processing model	Disk-based, map→shuffle→reduce	In-memory DAG of stages
Speed	Slower (disk I/O per stage)	Faster (memory + optimized DAG)
APIs	Java-heavy, verbose	Python/Scala/Java/R, concise
Use case	Cheap, huge batch jobs	Iterative, interactive, streaming

Key nuance: Spark is a **processing engine**, not a storage system. It often *replaces* *MapReduce* but frequently *reuses* HDFS or a cloud object store for storage.

Q4. ● What is PySpark?

PySpark is the Python API for Spark. It lets you write Spark applications in Python while the heavy lifting runs on the JVM. Python code is translated into JVM operations via the Py4J bridge, so you get Python's ease of use with Spark's distributed performance.

Watch-out: Python UDFs incur serialization overhead (data moves JVM → Python → JVM). Prefer built-in DataFrame functions or pandas UDFs when performance matters.

Q5. ● Explain Spark's architecture: driver, executors, and cluster manager.

- **Driver** — the process running your `main()`. It builds the DAG, splits it into stages/tasks, and schedules them. It hosts the `SparkContext/SparkSession`.

- **Executors** — worker processes on cluster nodes that run the actual tasks and hold cached data in memory.
- **Cluster Manager** — allocates resources (YARN, Kubernetes, Standalone, or Mesos). The driver asks it for executors.

Real-world example: Think of the driver as a construction site foreman: it reads the blueprint (your code), breaks the job into tasks, and hands them to workers (executors). The cluster manager is the staffing agency that supplies the workers.

Q6. ● What is the difference between local mode and cluster mode?

- **Local mode** — Spark runs entirely in one JVM on your machine (`local[*]` uses all cores). Great for development and unit tests; no cluster needed.
- **Cluster mode** — the application runs across multiple nodes managed by a cluster manager, giving true parallelism and scale.

Real-world example: A developer prototypes a transformation on a 10 MB sample in `local[4]` on a laptop, then submits the identical code to a Dataproc cluster to process the full 2 TB dataset. Same code, different scale.

Section 2 — RDDs (Resilient Distributed Datasets)

Q7. ● What is an RDD and what are its key properties?

An RDD is Spark's original low-level abstraction: an immutable, partitioned, distributed collection of records that can be processed in parallel. Key properties:

- **Resilient** — rebuilds lost partitions using lineage.
 - **Distributed** — partitioned across the cluster.
 - **Immutable** — transformations create new RDDs rather than modifying existing ones.
 - **Lazily evaluated** — nothing computes until an action is called.
-

Q8. ● How does an RDD achieve fault tolerance without replication?

Through **lineage** (the DAG of transformations that produced it). Spark records *how* each RDD was derived from its parent. If a partition is lost (node failure), Spark recomputes only that partition by replaying the lineage from the last available source — no need to replicate data across nodes like HDFS does.

Real-world example: A 200-node job loses one node mid-run. Instead of failing, Spark identifies the $\sim 1/200$ of partitions that lived on that node and recomputes just those from the source data, so the job completes with a minor delay instead of restarting.

Q9. ● What is the difference between a transformation and an action? Why does it matter?

- **Transformations** (map, filter, flatMap, reduceByKey) are **lazy** — they define a new RDD but don't execute.
- **Actions** (collect, count, saveAsTextFile, take) **trigger** execution and return a result or write output.

It matters because lazy evaluation lets Spark see the whole DAG before running and **optimize** it — for example, combining a filter and a map into a single pass, or pushing a filter earlier to reduce data volume.

Interview trap: “Does `rdd.map(...)` run immediately?” — No. It builds a plan. Only an action like `.count()` forces computation.

Q10. ● Explain narrow vs wide transformations and why wide ones are expensive.

- **Narrow** — each input partition contributes to exactly one output partition (map, filter, union). No data movement across the network.
- **Wide** — input partitions feed multiple output partitions, requiring a **shuffle** (groupByKey, reduceByKey, join, distinct). Data is written to disk and sent across the network, and a new **stage** boundary is created.

Wide transformations are expensive because shuffles involve disk I/O, network transfer, and serialization — usually the #1 performance bottleneck in Spark jobs.

Real-world example: A job doing groupByKey on 500 M records was shuffling the full dataset. Switching to reduceByKey — which aggregates locally on each partition *before* shuffling — cut shuffle volume by ~80% and halved runtime.

Q11. ● reduceByKey vs groupByKey — which is better and why?

reduceByKey is almost always better for aggregations. It performs **map-side combine**: it aggregates values within each partition *before* the shuffle, so far less data crosses the network. groupByKey shuffles *all* values first, then groups — risking large shuffles and out-of-memory errors when a key has many values.

Rule of thumb: If you're aggregating, reach for reduceByKey/aggregateByKey, not groupByKey.

Q12. ● What is a Key-Value Pair RDD and when do you use it?

A pair RDD is an RDD of (key, value) tuples. It unlocks by-key operations: `reduceByKey`, `groupByKey`, `sortByKey`, `join`, `countByKey`. You use it whenever your logic is keyed — aggregating sales by store, joining orders to customers, counting events per user.

Real-world example: To compute total revenue per store, map each transaction to (store_id, amount) and then `reduceByKey(lambda a, b: a + b)`.

Q13. ● How do you load and save data through RDDs?

- **Load:** `sc.textFile("gs://bucket/path")` reads text into an RDD of lines; `sc.wholeTextFiles(...)` reads whole files as (filename, content).
- **Save:** `rdd.saveAsTextFile("gs://bucket/output")` writes one file per partition.

Note that Spark writes a *directory* of part-files (part-00000, part-00001, ...), one per partition — not a single file. It will also error if the output path already exists, which prevents accidental overwrites.

Section 3 — Shared Variables

Q14. ● What are broadcast variables and why use them?

A broadcast variable ships a **read-only** copy of a value to every executor **once**, cached in memory, instead of shipping it with every task. Ideal for small lookup tables referenced across many tasks.

Real-world example: You need to enrich 1 B click records with a 5 MB country-code lookup. Without broadcasting, that lookup is serialized and sent with every task (thousands of times). Broadcast it once per executor, and each task reads it locally — big network savings.

Q15. ● What are accumulators and what's their main limitation?

Accumulators are **write-only-from-tasks, read-from-driver** variables used to aggregate information across the cluster — counters, sums, error tallies. Executors add to them; only the driver reads the final value.

Limitation: Reliable **only inside actions**. In transformations, a task can re-run (on failure or speculative execution) and double-count. Use them for monitoring/diagnostics (e.g., “how many malformed rows did we skip?”), not for business-critical results.

Real-world example: An ETL job uses an accumulator `bad_records` to count rows that fail schema validation, then logs “Skipped 4,213 malformed records” for the data-quality dashboard.

Section 4 — Cluster Deployment (Cluster Manager, EMR, Dataproc, spark-submit)

Q16. ● What is a cluster manager and what are the common options?

The cluster manager allocates resources (CPU/memory) to Spark applications and launches executors. Options: **Standalone** (Spark's built-in), **YARN** (Hadoop ecosystem — used by EMR and Dataproc), **Kubernetes**, and **Mesos** (legacy).

Q17. ● Explain client deploy mode vs cluster deploy mode. When would you choose each?

The difference is **where the driver runs**:

- **Client mode** — the driver runs on the machine that submitted the job (e.g., your gateway/edge node). You see logs directly. Good for interactive work and notebooks, but if that machine dies, the job dies.
- **Cluster mode** — the driver runs *inside the cluster* on a worker node. More resilient and standard for production/scheduled jobs, since the submitting machine can disconnect.

Real-world example: Data scientists run `spark-shell` in **client mode** for interactive exploration; the nightly production ETL is submitted in **cluster mode** so it survives the CI/CD runner shutting down after submission.

Q18. ● Walk through what `spark-submit` does.

`spark-submit` is the single entry point to launch Spark apps. It packages your application, sets configuration and resources, and hands it to the cluster manager. A typical call:

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --num-executors 10 \
  --executor-memory 8g \
  --executor-cores 4 \
  --driver-memory 4g \
  my_etl_job.py --input gs://bucket/in --output gs://bucket/out
```

It specifies the master (cluster manager), deploy mode, executor/driver resources, and your app plus its arguments.

Q19. ● What is an executor, and how do you size executor memory and cores?

An executor is a JVM process on a worker node that runs tasks and stores cached partitions. Sizing balances parallelism against overhead:

- **Cores per executor:** ~4–5 is a common sweet spot. Too many cores per executor causes HDFS/GCS I/O contention; too few wastes JVM overhead.
- **Memory:** leave headroom for OS and off-heap; Spark reserves part of executor memory for execution (shuffle/joins) and storage (cache).

Real-world example: On a 100-core cluster, “fat” executors (1 executor × 100 cores) serialize on I/O, while “tiny” executors (100 × 1 core) waste overhead. A balanced config — say 20 executors × 5 cores — usually performs best.

Q20. ● What is the “driver memory” and when does it become a bottleneck?

Driver memory is the heap for the driver process. It’s usually fine unless you pull large results back to the driver. The classic failure is calling `.collect()` on a huge dataset — that streams *all* data to the single driver JVM and causes an OOM.

Real-world example: A junior engineer ran `df.collect()` on a 50 GB result to “peek at the data,” crashing the driver. Fix: use `df.show(20)` or `df.limit(1000).collect()`, or write results to storage instead of collecting.

Q21. ● Contrast AWS EMR and GCP Dataproc.

Both are managed Hadoop/Spark cluster services. The concepts map almost 1:1:

Concept	AWS EMR	GCP Dataproc
Managed cluster	EMR cluster	Dataproc cluster
Object storage (HDFS replacement)	S3	GCS (Cloud Storage)
Submit a job	EMR Step / spark-submit	<code>gcloud dataproc jobs submit</code>
Identity/permissions	IAM roles	IAM roles + service accounts

Interview framing: “I’ve used EMR; on GCP the equivalent is Dataproc — same Spark, YARN under the hood, but I read/write GCS instead of S3 and submit with `gcloud dataproc jobs submit`.”

Q22. ● How do you submit a Spark job on Dataproc?

```
gcloud dataproc jobs submit pyspark \  
  --cluster=my-cluster \  
  --region=us-central1 \  
  --
```

```
gs://my-bucket/jobs/etl_job.py \  
-- --input gs://my-bucket/raw --output gs://my-bucket/curated
```

gcloud dataproc jobs submit wraps spark-submit, targets a named cluster and region, and passes app arguments after the -- separator.

Q23. ● Explain “GCS as the HDFS replacement.” What are the trade-offs?

On GCP, Cloud Storage (GCS) commonly replaces HDFS as the durable storage layer, accessed via the `gs://` connector. Benefits: **decoupled storage and compute** (spin clusters up/down without moving data), 11-nines durability, and lower cost than always-on HDFS.

Trade-offs to mention: - GCS is **object storage**, not a true filesystem — “rename/move” is a copy+delete, so naive commit operations can be slow. Use appropriate output committers. - **Eventual-consistency** concerns of the past are largely resolved (GCS is strongly consistent now), but list-heavy operations on huge directories can still be slow.

Real-world example: A team stores curated Parquet in GCS. Each morning they launch an **ephemeral** Dataproc cluster, run transformations reading/writing `gs://`, and delete the cluster — paying only for compute during the run, with data safely persisted in GCS.

Section 5 — GCP Foundations (Console/CLI, Cloud Storage, IAM)

Q24. ● What are Cloud Storage buckets and how is data organized?

A bucket is a top-level container for objects (files) in GCS, with a globally unique name. Objects are stored flat but use `/` in names to *simulate* folders. Buckets have a **storage class** (Standard, Nearline, Coldline, Archive) chosen by access frequency, and a **location** (regional/multi-regional).

Real-world example: A data lake uses zones as prefixes: `gs://acme-lake/raw/...`, `gs://acme-lake/staging/...`, `gs://acme-lake/curated/...` — the medallion pattern expressed as bucket paths.

Q25. ● Explain IAM roles for data workloads and the principle of least privilege.

IAM controls *who* (identity) can do *what* (role) on *which* resource. For data workloads you typically attach a **service account** to the Dataproc cluster or job, then grant it only the roles it needs — e.g., `roles/storage.objectViewer` on the raw bucket and `roles/storage.objectAdmin` on the output bucket, plus `roles/bigquery.dataEditor` if it writes to BigQuery.

Least privilege means granting the minimum permissions required — never Owner or Editor on the project just to “make it work.”

Real-world example: An ETL service account can *read* the raw landing bucket and *write* only to the curated bucket. Even if the job code were compromised, it can’t delete source data or touch unrelated datasets.

PART B — WEEK 2: Spark SQL, DataFrames, Datasets & BigQuery

Section 6 — Spark SQL & Entry Points

Q26. ● What is Spark SQL and what does it add over core RDDs?

Spark SQL is the module for structured data. It adds the **DataFrame/Dataset** APIs, lets you run SQL against your data, and — crucially — introduces the **Catalyst optimizer** and **Tungsten** execution engine, which optimize queries automatically (predicate pushdown, column pruning, code generation). RDDs are opaque to Spark; DataFrames carry schema so Spark can optimize.

Q27. ● SparkSession vs SparkContext — what’s the difference?

- **SparkContext** — the original entry point to Spark Core (RDDs), created since 1.x.
- **SparkSession** — the unified entry point introduced in Spark 2.0. It wraps SparkContext, SQLContext, and HiveContext into one object and is how you create DataFrames and run SQL.

Today you create a SparkSession; you can still reach the underlying context via `spark.sparkContext` when you need low-level RDD operations.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("etl").getOrCreate()
```

Q28. ● SQLContext vs HiveContext — why did SparkSession replace them?

- **SQLContext** — basic Spark SQL functionality.
- **HiveContext** — a superset adding Hive support: the Hive metastore, HiveQL, and Hive UDFs/window functions.

Before 2.0 you picked one (or juggled both). SparkSession unified them — enable Hive support with `.enableHiveSupport()` — so you no longer manage separate contexts.

Q29. ● What is a DataFrame and how does it differ from an RDD?

A DataFrame is a distributed collection of data organized into **named columns** — conceptually a table with a schema. Versus RDD:

- **Schema-aware** → Catalyst can optimize; RDDs cannot.
- **Higher-level API** (select, filter, groupBy) vs functional RDD ops.
- **Better performance** via columnar formats and code generation.

Rule of thumb: Use DataFrames/SQL by default; drop to RDDs only for low-level control you can't express with the structured API.

Q30. ● DataFrame vs Dataset — what's the trade-off?

- **Dataset** — a *typed* API (Scala/Java) giving **compile-time type safety** and object-oriented access, at the cost of some optimization/serialization overhead.
- **DataFrame** — in Scala it's an alias for Dataset[Row] (untyped). In **PySpark there is no Dataset API** — Python is dynamically typed, so you always work with DataFrames.

Interview nuance: If asked in a PySpark context, note that Datasets are a JVM (Scala/Java) concept; PySpark uses DataFrames and gets its safety at runtime, not compile time.

Section 7 — DataFrame Operations

Q31. ● How do you select, filter, and add/remove columns in a DataFrame?

```
df.select("name", "salary")           # select columns
df.filter(df.salary > 100000)          # filter rows (== where)
df.withColumn("bonus", df.salary * 0.1) # add/derive a column
df.drop("ssn")                         # remove a column
df.withColumnRenamed("dob", "birth_date") # rename
```

filter and where are aliases. withColumn returns a *new* DataFrame (immutability).

Q32. ● How do you work with JSON datasets in Spark?

```
df = spark.read.json("gs://bucket/events.json")
```

Spark **infers the schema** by scanning the data. For nested JSON, use dot notation (df.select("address.city")) and explode() to flatten arrays into rows.

Real-world example / watch-out: Schema inference reads the data twice (once to infer, once to load) and can misread types on messy data. For production pipelines, **supply an explicit schema** (StructType) — it's faster and prevents surprise type changes when new data arrives.

Q33. 🟡 Explain aggregate functions and `groupBy` with an example.

`groupBy` partitions rows by key, then aggregations (count, sum, avg, min, max) collapse each group to one row.

```
from pyspark.sql import functions as F
df.groupBy("region") \
    .agg(F.sum("revenue").alias("total_rev"),
         F.avg("order_value").alias("avg_order")) \
    .orderBy(F.desc("total_rev"))
```

Real-world example: Compute total and average revenue per region for a weekly executive dashboard — one `groupBy().agg()` replaces a hand-written multi-pass loop.

Q34. 🟡 What set operations does Spark support?

- `union` / `unionByName` — stack rows (append). `union` matches by position; `unionByName` matches by column name (safer).
- `intersect` — rows present in both.
- `subtract` / `exceptAll` — rows in the first not in the second.

Watch-out: `union` does **not** dedupe by default and aligns by column order — mismatched schemas silently corrupt data. Prefer `unionByName`.

Section 8 — Joins

Q35. 🟢 What join types does Spark support?

`inner`, `left` (left outer), `right` (right outer), `full` (full outer), `left_semi` (rows in left that have a match — no right columns), `left_anti` (rows in left with **no** match — great for “find missing”), and `cross`.

```
orders.join(customers, on="customer_id", how="left")
```

Q36. 🔴 What is a broadcast join and when does Spark use it?

In a broadcast (map-side) join, the **small** table is broadcast to every executor so the join happens locally — **no shuffle** of the large table. Spark auto-broadcasts tables under `spark.sql.autoBroadcastJoinThreshold` (default 10 MB), or you can force it:

```
from pyspark.sql.functions import broadcast
large_df.join(broadcast(small_df), "key")
```

Real-world example: Joining a 2 TB fact table to a 30 MB “product dimension.” A normal shuffle join would reshuffle 2 TB across the network; broadcasting the 30 MB dimension eliminates that shuffle and can turn a 40-minute job into a few minutes.

Q37. ● What is data skew in a join and how do you mitigate it?

Skew happens when a few keys hold a disproportionate share of rows (e.g., null or a “default” customer). Those keys create huge partitions, so one or two tasks run far longer than the rest — the whole stage waits on stragglers.

Mitigations: enable **Adaptive Query Execution** (AQE handles skew joins automatically in Spark 3+), **salt** the hot keys (append a random suffix to spread them across partitions), filter/handle nulls separately, or broadcast if one side is small.

Real-world example: 60% of transactions had `store_id = NULL` (online orders). One task processed all of them and hung the job. Splitting nulls into a separate branch and unioning the results balanced the load.

Section 9 — Partitioning, Sorting, Bucketing & Caching

Q38. ● repartition vs coalesce — what’s the difference?

- **repartition(n)** — full shuffle; can increase *or* decrease partitions; produces evenly sized partitions.
- **coalesce(n)** — narrow, avoids a full shuffle; used to **decrease** partitions by merging adjacent ones; may produce uneven sizes.

Rule of thumb: Use coalesce to shrink partitions cheaply (e.g., before writing fewer output files); use repartition when you need balance or more partitions.

Real-world example: An output writing 2,000 tiny files was hard on downstream reads. `coalesce(50)` before write produced 50 reasonably sized files without a costly reshuffle.

Q39. ● Explain partitioning-on-write and why partition pruning matters.

`df.write.partitionBy("event_date")` physically splits output into subdirectories per value (`event_date=2026-07-25/...`). Later, a query filtered on that column reads **only** the relevant folders — **partition pruning** — skipping the rest.

Real-world example: A 3-year event table partitioned by date. `WHERE event_date = '2026-07-24'` scans one day’s folder instead of three years of data — orders of magnitude less I/O and cost.

Watch-out: Don't partition on high-cardinality columns (e.g., `user_id`) — it creates millions of tiny folders (the “small files problem”). Partition on low/medium-cardinality columns you actually filter on.

Q40. ● What is bucketing and how does it differ from partitioning?

Bucketing hashes a column into a fixed number of buckets (files) — `df.write.bucketBy(50, "customer_id")`. Unlike partitioning (one folder per value, best for low cardinality), bucketing suits **high-cardinality** join/aggregation keys. Two tables bucketed the same way on the join key can be joined **without a shuffle**.

Real-world example: Two large tables repeatedly joined on `customer_id`. Bucketing both by `customer_id` into 200 buckets removed the shuffle from every subsequent join — a big win for a job that ran hourly.

Q41. ● Explain Spark caching and the common storage levels.

`cache()` / `persist()` keep a DataFrame in memory (and/or disk) so repeated actions don't recompute the lineage. `cache() = persist(MEMORY_AND_DISK)` for DataFrames. Levels include `MEMORY_ONLY`, `MEMORY_AND_DISK` (spill to disk if it doesn't fit), and `DISK_ONLY`.

When it helps: a DataFrame reused across multiple actions/branches. *When it hurts:* caching something used once wastes memory. Remember caching is **lazy** — it materializes on the first action — and to `unpersist()` when done.

Real-world example: A cleaned DataFrame feeds three separate reports. Without caching, Spark recomputes the full cleaning pipeline three times. `df.cache()` computes it once and reuses it, cutting total runtime by ~two-thirds.

Section 10 — BigQuery & Spark-vs-BigQuery Decisions

Q42. ● What is BigQuery and how is it architecturally different from Spark?

BigQuery is GCP's **serverless** data warehouse. You write SQL; Google manages all infrastructure — no clusters to size or start. It separates storage (Colossus) from compute (Dremel), scaling compute automatically. Spark is a general-purpose engine *you* provision and tune (via Dataproc); BigQuery is a fully managed SQL query service.

Q43. ● Explain partition pruning and clustering in BigQuery.

- **Partitioning** — split a table by a column (usually a date/timestamp or ingestion time). Queries filtering on the partition column scan only matching partitions → less data scanned, lower cost.
- **Clustering** — sorts data *within* partitions by up to four columns. Think of it as **column-level pre-sorting**: filters/aggregations on clustered columns read fewer blocks.

Real-world example: An events table partitioned by event_date and clustered by country. WHERE event_date = '2026-07-24' AND country = 'US' prunes to one partition, then uses clustering to skip non-US blocks — dramatically cutting bytes scanned (and cost, since on-demand BigQuery bills per byte scanned).

Q44. ● Slot-based execution (BigQuery) vs the Spark executor model — contrast them.

- **Spark executors** — you explicitly provision executors (count, cores, memory). You own capacity and tuning; idle clusters still cost money unless torn down.
- **BigQuery slots** — a slot is a unit of compute (a virtual CPU) that BigQuery assigns to your query automatically. You don't manage machines; BigQuery allocates slots from a shared or reserved pool and parallelizes the query for you.

The mental shift: with Spark you think in *executors/cores/memory*; with BigQuery you think in *slots and bytes scanned*.

Q45. ● On-demand vs reserved (flat-rate) capacity in BigQuery — when do you choose each?

- **On-demand** — pay per **bytes scanned**; slots allocated dynamically (with a fair-use cap). Best for spiky or unpredictable workloads and getting started.
- **Reserved / capacity-based (editions)** — buy a committed pool of **slots** for a predictable monthly cost, shared across queries. Best for heavy, steady workloads where per-byte billing gets expensive or where you need guaranteed capacity/predictable spend.

Real-world example: A startup runs a few ad-hoc analyses a day → **on-demand** is cheapest. A mature company running thousands of scheduled dashboards and pipelines hits predictable, heavy volume → **reserved slots** cap and stabilize the bill and prevent one team's huge scan from spiking costs.

Q46. ● When would you choose BigQuery over Spark SQL, and vice versa?

Choose BigQuery when: - The workload is primarily SQL analytics / BI / ad-hoc querying.
- You want zero infrastructure management (serverless). - Data already lives in BigQuery or GCS and answers come from aggregations.

Choose Spark (Dataproc) when: - You need complex, procedural transformations, custom code, ML pipelines, or non-SQL logic (Python/Scala). - You're doing large-scale ETL with libraries, streaming, or fine-grained control over processing. - You need to process semi/unstructured data before it's warehouse-ready.

Real-world pattern (both, not either/or): Use **Spark on Dataproc** to clean and transform raw files landing in GCS, write curated Parquet, then load into **BigQuery** where analysts run fast SQL and Looker dashboards. Spark does the heavy transformation; BigQuery serves the analytics.

Rapid-Fire Round (test yourself in 30 seconds each)

1. ● Is a transformation lazy or eager? → *Lazy*.
2. ● Name two actions and two transformations. → *Actions: count, collect. Transformations: map, filter.*
3. ● Which causes a shuffle: map or reduceByKey? → *reduceByKey (wide)*.
4. ● Default broadcast join threshold? → *10 MB*.
5. ● cache() equals which persist level for a DataFrame? → *MEMORY_AND_DISK*.
6. ● Does PySpark have a Dataset API? → *No — DataFrames only*.
7. ● GCS equivalent of an EMR Step? → *gcloud dataproc jobs submit*.
8. ● BigQuery on-demand bills by...? → *Bytes scanned*.
9. ● repartition vs coalesce — which shuffles? → *repartition*.
10. ● Best fix for join data skew in Spark 3+? → *Enable AQE (and/or salt keys)*.

Interviewer's Notes (how answers are scored)

As a program manager reviewing associate mock interviews, strong answers consistently:

- **Lead with the concept in one sentence**, then give the *why*, then a concrete example.
- **Name the trade-off**. "Broadcast join avoids a shuffle, *but* only when one side fits in memory" scores higher than a one-sided answer.
- **Translate across clouds**. Show you understand EMR ↔ Dataproc, S3 ↔ GCS map onto the same Spark concepts.
- **Reach for a real number**. "We cut a 40-minute job to 4 minutes" lands better than "it was faster."
- **Admit boundaries**. For "BigQuery vs Spark," the best answer is often "both, in a pipeline" — not picking a side.

Copyright note: This study material is derived from the Revature curriculum outline you provided. Keep distribution within the terms of your program agreement.